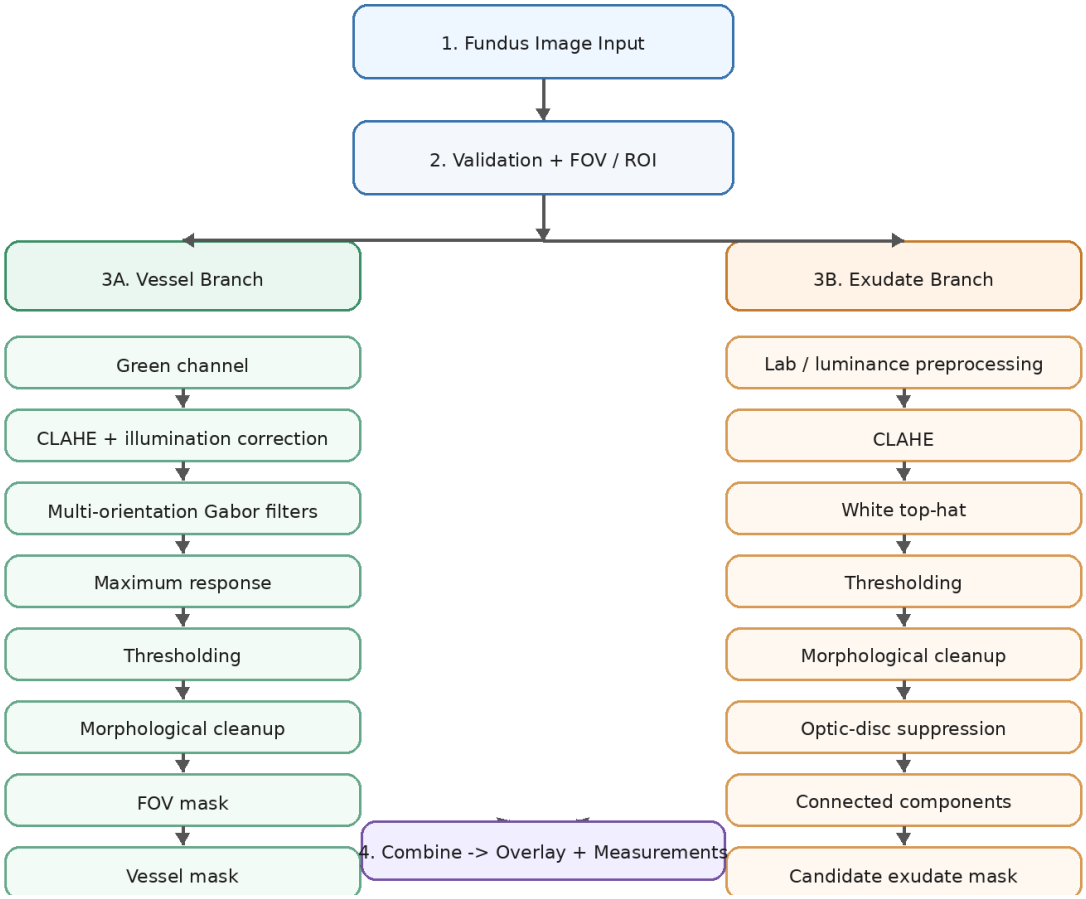
RETINAL IMAGE ANALYSIS PLATFORM

Complete Project Build Guide

Retinal Blood Vessel Segmentation & Candidate Hard-Exudate Detection Using Classical Digital Image Processing

Retinal DIP Platform - End-to-End Architecture



Python + OpenCV + NumPy + scikit-image + Matplotlib + Streamlit + ReportLab

Project rule: This guide is organized as a checkpoint-based build. Do not build the Streamlit interface first. First make the image-processing pipeline correct and measurable, then integrate the UI and report.

Prepared as an academic / research prototype guide. Not a clinical diagnostic workflow.

Contents and how to use this guide

1. 1. Project definition and final deliverables
2. 2. What each stage means in simple terms
3. 3. Dataset strategy: DRIVE and IDRiD
4. 4. Development environment and installation
5. 5. Final project folder structure
6. 6. Build strategy and milestone checkpoints
7. 7. Stage A - Load, inspect, and validate images
8. 8. Stage B - Preprocessing
9. 9. Stage C - Blood vessel segmentation with Gabor filters
10. 10. Stage D - Candidate hard-exudate detection
11. 11. Stage E - Candidate region analysis
12. 12. Stage F - Evaluation and metrics
13. 13. Stage G - Experiments and parameter tuning
14. 14. Stage H - Integrate the complete pipeline
15. 15. Stage I - Build the Streamlit application
16. 16. Stage J - Generate the PDF report
17. 17. Stage K - Testing, debugging, and failure cases
18. 18. Stage L - Documentation, PPT, demo, and viva
19. 19. Troubleshooting guide
20. 20. Final success checklist
21. 21. References

> **How to use this guide:** Use the acceptance checks after each stage. If a stage fails visually or numerically, stop and fix it before adding another stage. This prevents a large codebase from hiding a small image-processing error.

1. Project definition and final deliverables

Project title: Retinal Blood Vessel Segmentation & Candidate Hard-Exudate Detection Using Classical Digital Image Processing.

A retinal fundus photograph is the input. The system performs two image-analysis branches: (1) blood-vessel segmentation and (2) candidate hard-exudate detection. The outputs are combined into an annotated image and quantified with measurable image features. When annotated ground truth exists, the same pipeline is evaluated using segmentation metrics.

1.1 Final outputs

- Enhanced retinal image after preprocessing.
- Binary blood-vessel mask.
- Binary candidate hard-exudate mask.
- Combined overlay showing detected vessels and candidate lesion regions.
- Candidate-region table with area, centroid, and bounding box.
- Vessel density measurement within the field of view.
- Pixel-level evaluation metrics when a ground-truth mask is available.
- Downloadable PDF analysis report.
- Streamlit web interface for interactive demonstration.

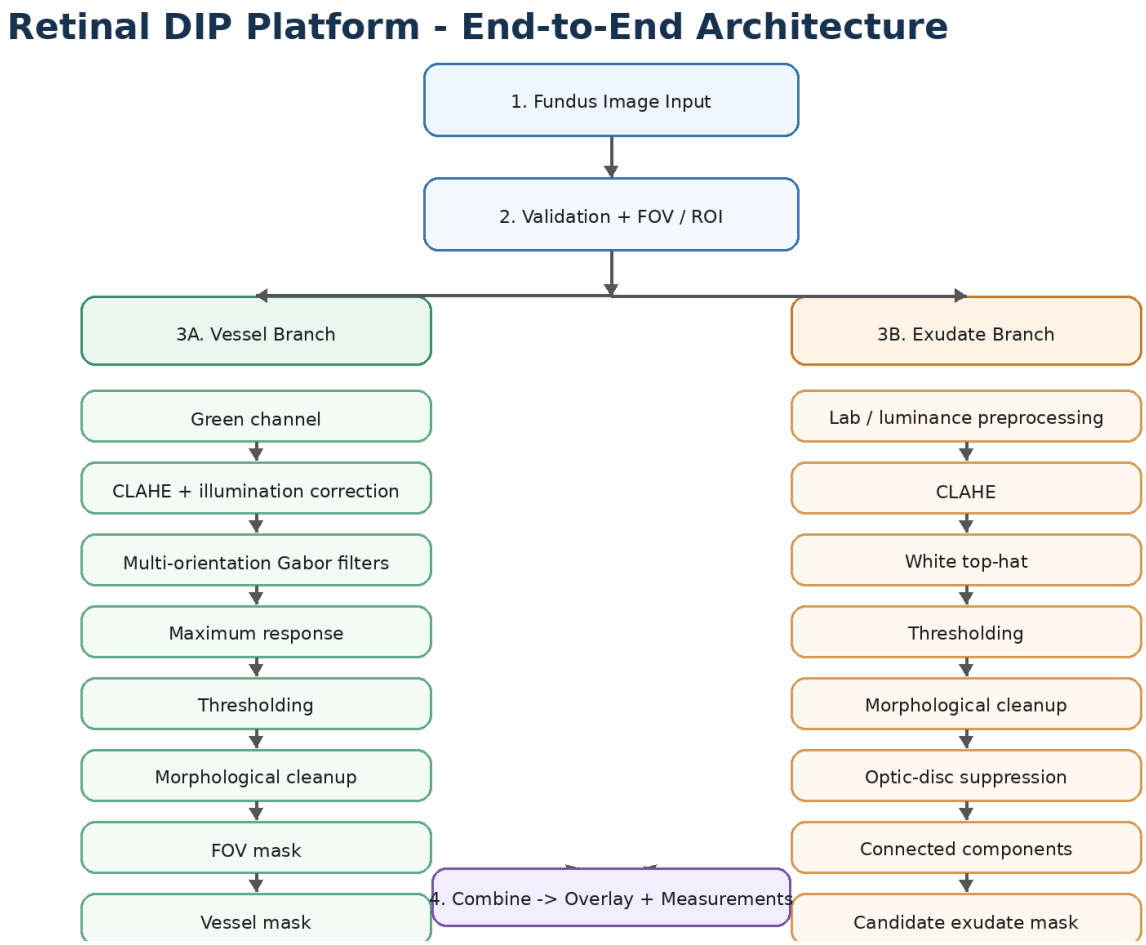
1.2 What the project must NOT claim

- Do not state that the system diagnoses diabetic retinopathy.
- Do not convert a candidate bright region into a confirmed clinical lesion.

- Do not display invented metrics, patient data, or clinical conclusions.
- Do not use test-set images to tune thresholds and then report that same result as unbiased validation.
- Do not use real patient photographs for development unless appropriate consent, privacy controls, and institutional procedures are in place.

Safe project language: Preferred wording in the application: “Candidate hard-exudate regions detected. Clinical review is recommended where appropriate.” Add a visible “Research / decision-support prototype; not a medical diagnosis” disclaimer.

1.3 Final system architecture



2. What each stage means in simple terms

Stage	Simple meaning	Main DIP idea
Input	Read a retinal image and identify usable pixels.	Image I/O, validation
Preprocessing	Make faint structures easier to see.	Color channel selection, CLAHE, smoothing, background correction
Vessel branch	Highlight long, thin vessel-like structures.	Gabor filtering, thresholding, morphology
Exudate branch	Highlight bright local regions and turn them into candidates.	Top-hat morphology, thresholding, connected components
Post-processing	Remove obvious noise and restrict analysis to the retina.	FOV masking, morphology
Measurements	Turn masks into numbers.	Pixel counting, connected components
Evaluation	Compare predictions to annotation.	TP/FP/FN/TN, Dice, IoU, sensitivity,

		specificity, precision, F1
UI	Let a person upload and inspect results.	Streamlit
Report	Package images, measurements and metrics.	ReportLab

2.1 The mental model

Think of the entire project as “make the target structure easier to see, isolate it, clean it, measure it, and compare it with known ground truth.” Every operation should have a reason.

Operation	Question it answers
Green channel	Which color channel gives useful vessel/background contrast?
CLAHE	Can local contrast be improved without relying on one global brightness level?
Gaussian blur	What does the slowly varying background illumination look like?
Gabor filters	Where are oriented, line-like structures?
Threshold	Which pixels have a strong enough response to become a candidate?
Opening	Which tiny isolated objects can be removed?
Closing	Which small gaps can be connected?
White top-hat	Which bright local structures stand out from their neighborhood?
Connected components	Which pixels form an individual candidate region?
Dice / IoU	How much does the prediction overlap the annotation?

3. Dataset strategy: DRIVE and IDRiD

The project intentionally uses two datasets because the segmentation targets are different.

3.1 DRIVE - vessel segmentation

The official DRIVE site describes 40 retinal photographs, divided into 20 training images and 20 test images. Images are 768 x 584 pixels, use a 45-degree field of view, and each image has a field-of-view mask. A manual vessel segmentation is provided for the training images; test annotations are not publicly provided on the site. The official evaluation calculates Dice per image and considers only pixels within the provided mask. [1]

Property	DRIVE
Purpose	Retinal blood-vessel segmentation
Total images	40
Training / test	20 / 20
Image size	768 x 584
FOV	Approx. 45 degrees; circular FOV
ROI mask	Provided for each image
Training vessel annotation	Available
Test vessel annotation	Not publicly provided on the site
Official Dice evaluation	Yes, within the mask

3.2 IDRiD - hard exudate analysis

The official IDRiD data page states that 516 images were extracted for the dataset. The pixel-level annotated subset contains 81 color fundus images with masks for hard exudates (EX), microaneurysms (MA),

hemorrhages (HE), and soft exudates (SE); the number of images with each lesion mask is 81, 81, 80, and 40 respectively. Optic-disc masks are provided for all 81 pixel-level images, and optic-disc and fovea-center locations are also supplied separately. [2]

Property	IDRiD
Total dataset	516 fundus images
Pixel-level lesion subset	81 images
Hard-exudate masks	81 images
Optic-disc masks	81 images
Fovea coordinates	Provided separately
Image format	JPEG fundus photographs
Reported image resolution	4288 x 2848
Camera / FOV	Kowa VX-10 alpha; 50-degree FOV

> **Dataset design:** For the core project, use DRIVE to demonstrate vessel segmentation and IDRiD to demonstrate hard-exudate candidate detection. In your report, explicitly explain why one dataset is not being forced to cover both tasks.

3.3 Dataset folders

```
data/
├── DRIVE/
│   ├── training/
│   │   ├── images/
│   │   ├── fov_masks/
│   │   └── vessel_masks/
│   └── test/
│       ├── images/
│       └── fov_masks/
│
└── IDRiD/
    ├── images/
    ├── exudate_masks/
    ├── optic_disc_masks/
    └── metadata/
```

3.4 Dataset handling rules

- Never silently resize a ground-truth mask differently from its corresponding prediction.
- Keep original files untouched; create processed copies in outputs/.
- Keep image IDs consistent across prediction and ground truth.
- Use the FOV mask during vessel evaluation on DRIVE.
- For arbitrary user uploads, use a fallback FOV detector if a dataset mask is not available.
- For IDRiD evaluation, keep the provided optic-disc mask available for an evaluation-mode suppression experiment.

4. Development environment and installation

4.1 Recommended stack

Tool	Role
Python	Main programming language
OpenCV	Image I/O, filtering, morphology, thresholding, connected components
NumPy	Numerical arrays and pixel operations
scikit-image	Optional image-processing utilities
Matplotlib	Intermediate visual inspection and experiment plots

Pandas	Result tables and CSV analysis
Streamlit	Interactive UI
ReportLab	PDF report generation

4.2 Create the environment

```
python -m venv venv

# Windows
venv\Scripts\activate

# Linux / macOS
source venv/bin/activate

pip install opencv-python numpy matplotlib scikit-image streamlit pandas reportlab
pip freeze > requirements.txt
```

4.3 requirements.txt baseline

```
opencv-python
numpy
matplotlib
scikit-image
streamlit
pandas
reportlab
```

Keep the stack focused: Do not install a large machine-learning stack just because it is common in medical imaging. The core project is intentionally based on classical DIP.

4.4 First environment test

```
import cv2
import numpy as np
import matplotlib
import skimage
import streamlit
import pandas
import reportlab

print("OpenCV:", cv2.__version__)
print("NumPy:", np.__version__)
print("Environment OK")
```

4.5 Success checkpoint

- Virtual environment activates without errors.
- All imports succeed.
- OpenCV version prints.
- A small test image can be read and displayed.
- No package import errors remain before moving to the dataset stage.

5. Final project folder structure

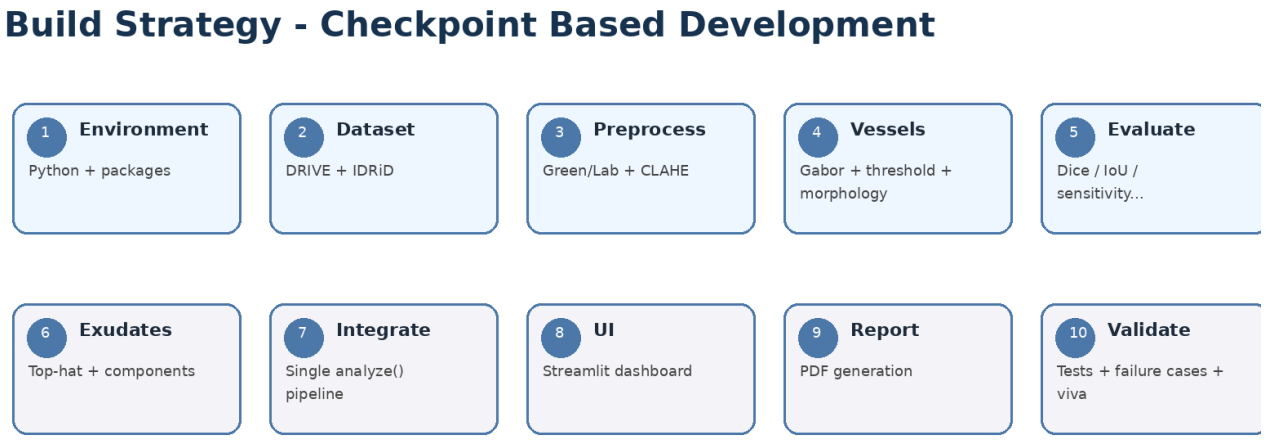
```
retinal-dip/
│
├── data/
│   ├── DRIVE/
│   │   ├── training/
│   │   │   ├── images/
│   │   │   ├── fov_masks/
│   │   │   └── vessel_masks/
```

```
│   │   └── test/
│   │       ├── images/
│   │       └── fov_masks/
│   │
│   └── IDRiD/
│       ├── images/
│       ├── exudate_masks/
│       ├── optic_disc_masks/
│       └── metadata/
│
├── src/
│   ├── __init__.py
│   ├── preprocessing.py
│   ├── vessels.py
│   ├── exudates.py
│   ├── morphology.py
│   ├── evaluation.py
│   ├── visualization.py
│   └── report.py
│
├── outputs/
│   ├── preprocessed/
│   ├── vessels/
│   ├── exudates/
│   ├── overlays/
│   └── reports/
│
├── experiments/
│   ├── results.csv
│   ├── vessel_experiments.csv
│   └── exudate_experiments.csv
│
├── app.py
├── main.py
├── requirements.txt
└── README.md
```

5.1 Purpose of each file

File	Responsibility
preprocessing.py	Color conversion, CLAHE, illumination correction, input normalization
vessels.py	Gabor bank, response aggregation, thresholding, morphology, FOV masking
exudates.py	Bright-region enhancement, top-hat, thresholding, optic-disc handling, components
morphology.py	Reusable opening/closing/kernel helpers
evaluation.py	Confusion matrix and segmentation metrics
visualization.py	Mask overlays, contour drawings, pipeline figures
report.py	PDF report creation
main.py	Command-line / batch pipeline
app.py	Streamlit UI

6. Build strategy and milestone checkpoints



Rule: after every milestone, save an image output and an evaluation/debug result before moving forward.

6.1 The 10 checkpoints

Checkpoint	Deliverable	Do not move on until
1	Environment test	All packages import successfully.
2	Dataset viewer	You can open image + mask pairs correctly.
3	Preprocessing	Green/Lab enhanced images look sensible.
4	Vessel mask	A vessel mask and overlay are produced for a sample.
5	Vessel evaluation	Metrics are computed correctly on known masks.
6	Exudate mask	Candidate bright regions are produced without crashing.
7	Region analysis	Candidates have area, centroid and bounding box.
8	Integrated pipeline	One function returns all outputs for one image.
9	Streamlit + report	Upload -> analyze -> result -> PDF works.
10	Final validation	Batch run, failure cases, documentation and viva material are complete.

Build discipline: A common reason student projects become unstable is combining 10 stages before testing the first 2. This guide deliberately avoids that.

7. Stage A - Load, inspect, and validate images

7.1 Read an image

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("path/to/image.tif")
if img is None:
```

```
    raise FileNotFoundError("Image could not be loaded")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.show()
```

7.2 Inspect shape and dtype

```
print("Shape:", img.shape)
print("Dtype:", img.dtype)
print("Min:", img.min())
print("Max:", img.max())
```

7.3 What you are checking

- Is this actually a fundus image?
- Is the color channel order correct?
- Are there black borders or an obvious FOV boundary?
- Is the image completely blank or corrupted?
- Are there extreme highlights or compression artifacts?
- Are image and ground-truth dimensions identical where evaluation is required?

7.4 Input validation function

```
def validate_image(image):
    if image is None:
        return False, "No image"

    if len(image.shape) != 3 or image.shape[2] != 3:
        return False, "Expected a 3-channel color image"

    h, w = image.shape[:2]
    if h < 100 or w < 100:
        return False, "Image is too small"

    return True, "OK"
```

7.5 FOV handling

For DRIVE, use the provided FOV mask in benchmark evaluation. For a generic uploaded image with no mask, a simple fallback can be created by thresholding non-black pixels, cleaning the binary result, and retaining the largest connected component. Treat this fallback as a practical application heuristic, not as a benchmark-standard annotation.

```
def estimate_fov(image):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    mask = np.where(gray > 10, 255, 0).astype(np.uint8)

    kernel = np.ones((9, 9), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

    n, labels, stats, _ = cv2.connectedComponentsWithStats(mask, 8)
    if n <= 1:
        return mask

    largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
    return np.where(labels == largest, 255, 0).astype(np.uint8)
```

7.6 Acceptance test

- Open one DRIVE image and its FOV mask side by side.
- Overlay the FOV mask and confirm it follows the retinal field.
- Open one IDRiD image and one EX mask.
- Confirm the EX mask is binary and aligns pixel-for-pixel with the image.

8. Stage B - Preprocessing

8.1 Goal

Preprocessing does not perform final segmentation. It improves the input so that later filters have a cleaner signal.

8.2 Vessel preprocessing pipeline

```
RGB fundus image
    -> Green channel
    -> CLAHE
    -> Background estimate
    -> Illumination-corrected image
```

8.3 Extract green channel

```
b, g, r = cv2.split(image_rgb)
green = g
```

8.4 CLAHE

```
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

enhanced = clahe.apply(green)
```

CLAHE is local contrast enhancement with clipping to reduce excessive amplification of noise. The exact parameters are not sacred; treat them as experiment variables.

8.5 Background / illumination correction

```
background = cv2.GaussianBlur(
    enhanced,
    (0, 0),
    sigmaX=15
)

corrected = cv2.subtract(enhanced, background)
corrected = cv2.normalize(
    corrected,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
```

Preprocessing check: The purpose is to reduce slow brightness variation, not to erase the retinal structures you want to detect. Inspect the result visually; if vessels disappear, the correction is too aggressive.

8.6 Exudate preprocessing

```
lab = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2LAB)
L, A, B = cv2.split(lab)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
L_enhanced = clahe.apply(L)
```

8.7 Preprocessing checkpoint

Create a 4-panel figure: original image, green channel, CLAHE result, and illumination-corrected result. Save it in outputs/preprocessed/. If the four panels look sensible, freeze preprocessing for the first baseline experiment and only tune one parameter at a time later.

9. Stage C - Blood vessel segmentation with Gabor filters

9.1 Why this branch works

Retinal vessels are approximately elongated, oriented structures. A Gabor filter is an oriented band-pass filter whose response changes with orientation. Instead of choosing one angle, use a bank of angles and keep the strongest response at each pixel.

9.2 Gabor parameters

Parameter	Meaning	Start value / experiment
Kernel size	Spatial support of the filter	21 x 21
sigma	Gaussian envelope spread	4
theta	Orientation	0 to 165 degrees, every 15 degrees
lambda	Wavelength	10
gamma	Aspect ratio	0.5
psi	Phase	Start with 0 or test pi/2
dtype	Kernel precision	CV_32F

9.3 Gabor filter bank

```
def gabor_max_response(image):
    image_f = image.astype(np.float32)
    max_response = np.zeros_like(image_f)

    for theta in np.arange(0, np.pi, np.pi / 12):
        kernel = cv2.getGaborKernel(
            (21, 21),
            sigma=4,
            theta=theta,
            lambd=10,
            gamma=0.5,
            psi=0,
            ktype=cv2.CV_32F
        )

        response = cv2.filter2D(
            image_f,
            cv2.CV_32F,
            kernel
        )

        # Absolute response makes the baseline robust to response sign.
        response = np.abs(response)
```

```
        max_response = np.maximum(max_response, response)

    return max_response
```

9.4 Normalize response

```
response_norm = cv2.normalize(
    max_response,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
```

9.5 Threshold

```
_, vessel_binary = cv2.threshold(
    response_norm,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

9.6 Morphological cleanup

```
kernel = np.ones((3, 3), np.uint8)

opened = cv2.morphologyEx(
    vessel_binary,
    cv2.MORPH_OPEN,
    kernel
)

closed = cv2.morphologyEx(
    opened,
    cv2.MORPH_CLOSE,
    kernel
)
```

9.7 Apply FOV mask

```
vessel_mask = cv2.bitwise_and(
    closed,
    closed,
    mask=fov_mask
)
```

9.8 Create vessel overlay

```
overlay = image_rgb.copy()
overlay[vessel_mask > 0] = [0, 255, 0]
```

9.9 Vessel-stage acceptance test

- Gabor response should show line-like retinal structures, not only background.
- The vessel mask should be mostly inside the FOV.
- Large black borders should not become vessels.
- The overlay should visibly follow major vessel trunks.
- No metric is accepted until the prediction and ground truth have been confirmed to be correctly aligned.

Honest evaluation: Do not promise a particular Dice value before running the experiment. A classical baseline can be useful even when it has visible false positives or misses fine vessels; those failures become material for the analysis section.

10. Stage D - Candidate hard-exudate detection

10.1 Goal

The goal is to identify bright, locally prominent regions that are candidates for hard exudates. Because the optic disc and other bright structures can also satisfy this rule, the output is intentionally called a candidate mask.

10.2 Baseline pipeline

```
RGB
 -> Lab
 -> L channel
 -> CLAHE
 -> White top-hat
 -> Threshold
 -> Morphological cleanup
 -> Optic-disc handling
 -> Connected components
 -> Candidate regions
```

10.3 White top-hat

White top-hat morphology is defined as the input image minus its morphological opening. It emphasizes bright structures that are smaller than the structuring element.

```
kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE,
    (15, 15)
)

tophat = cv2.morphologyEx(
    L_enhanced,
    cv2.MORPH_TOPHAT,
    kernel
)
```

10.4 Thresholding

```
_, candidate_binary = cv2.threshold(
    tophat,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

10.5 Morphology

```
kernel = np.ones((3, 3), np.uint8)
clean = cv2.morphologyEx(
    candidate_binary,
    cv2.MORPH_OPEN,
    kernel
)
clean = cv2.morphologyEx(
    clean,
    cv2.MORPH_CLOSE,
    kernel
)
```

10.6 Optic-disc problem

The optic disc is a bright retinal structure. A simple bright-region detector can therefore confuse it with an exudate. IDRiD provides optic-disc masks for the 81 pixel-level lesion images, which makes an evaluation-mode optic-disc suppression experiment possible. [2]

```
# Evaluation mode only when the annotation is available.
clean[optic_disc_mask > 0] = 0
```

Important distinction: For a live upload, a ground-truth optic-disc mask is not available. Later, add an automatic optic-disc localization module or label the benchmark-only suppression clearly in the UI and report.

10.7 Candidate components

```
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    clean,
    connectivity=8
)
```

10.8 Area filtering

```
MIN_AREA = 20  # starting point only; tune on development data

candidate_mask = np.zeros_like(clean)
regions = []

for i in range(1, num_labels):
    x = stats[i, cv2.CC_STAT_LEFT]
    y = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]
    area = stats[i, cv2.CC_STAT_AREA]
    cx, cy = centroids[i]

    if area >= MIN_AREA:
        candidate_mask[labels == i] = 255
        regions.append({
            "id": len(regions) + 1,
            "area": int(area),
            "bbox": [int(x), int(y), int(w), int(h)],
            "centroid": [float(cx), float(cy)]
        })
```

10.9 Candidate-region overlay

```
overlay = image_rgb.copy()

for r in regions:
    x, y, w, h = r["bbox"]
    cv2.rectangle(
        overlay,
        (x, y),
        (x + w, y + h),
        (255, 255, 0),
        2
    )
```

10.10 Why this is candidate detection, not diagnosis

- Top-hat detects bright local structures, not the semantic concept of a disease lesion.
- Large or irregular bright artifacts can become false positives.
- Some real lesions may be missed if their contrast is weak.
- Clinical confirmation requires domain expertise and is outside this academic pipeline.

11. Stage E - Candidate region analysis

11.1 What connected components give us

A binary candidate mask is useful visually, but connected components convert it into a list of objects. Each object can be measured independently.

Feature	Meaning	Use in project
Area	Number of pixels in the component	Largest / total candidate burden
Centroid	Geometric center	Localization on the retina
Bounding box	x, y, width, height	Visual annotation
Count	Number of retained components	Image-level summary
Distance to fovea	Distance from centroid to fovea center	Optional spatial analysis
Overlap with optic disc	Pixels inside optic-disc area	False-positive analysis

11.2 Candidate statistics

```
total_area = sum(r["area"] for r in regions)
largest_area = max((r["area"] for r in regions), default=0)
count = len(regions)

average_area = total_area / count if count else 0
```

11.3 Vessel density

```
vessel_pixels = np.sum(vessel_mask > 0)
fov_pixels = np.sum(fov_mask > 0)

vessel_density = 100.0 * vessel_pixels / (fov_pixels + 1e-8)
```

Report this as an image-derived descriptive feature, not as a clinically validated threshold.

11.4 Distance to fovea

```
import math

def distance_to_fovea(cx, cy, fx, fy):
    return math.sqrt((cx - fx) ** 2 + (cy - fy) ** 2)
```

IDRiD provides fovea-center coordinates separately, which can support a spatial analysis experiment. [2]

11.5 Recommended region table

ID	Area (px)	Centroid (x,y)	Bounding box	Fovea distance (px)
1	actual	actual	actual	optional
2	actual	actual	actual	optional
...	...	...	...	...

No invented numbers: Use “actual” placeholders only in the design template. In the final report, every numeric result must come from the executed pipeline.

12. Stage F - Evaluation and metrics

12.1 Pixel-level confusion matrix

	Ground truth positive	Ground truth negative
Prediction positive	TP	FP
Prediction negative	FN	TN

12.2 Dice coefficient

Dice measures overlap between prediction and ground truth:

```
Dice = 2 * TP / (2 * TP + FP + FN)


def dice_score(pred, gt):
    pred = pred > 0
    gt = gt > 0

    intersection = np.logical_and(pred, gt).sum()
    denom = pred.sum() + gt.sum()

    return (2.0 * intersection) / (denom + 1e-8)
```

12.3 IoU

```
IoU = TP / (TP + FP + FN)


def iou_score(pred, gt):
    pred = pred > 0
    gt = gt > 0

    inter = np.logical_and(pred, gt).sum()
    union = np.logical_or(pred, gt).sum()

    return inter / (union + 1e-8)
```

12.4 Sensitivity / recall

```
Sensitivity = TP / (TP + FN)
```

12.5 Specificity

```
Specificity = TN / (TN + FP)
```

12.6 Precision

```
Precision = TP / (TP + FP)
```

12.7 F1

```
F1 = 2 * Precision * Sensitivity / (Precision + Sensitivity)
```

12.8 Complete metric function

```
def calculate_metrics(pred, gt):
    pred = pred > 0
    gt = gt > 0
```

```
    tp = np.logical_and(pred, gt).sum()
    fp = np.logical_and(pred, ~gt).sum()
    fn = np.logical_and(~pred, gt).sum()
    tn = np.logical_and(~pred, ~gt).sum()

    precision = tp / (tp + fp + 1e-8)
    sensitivity = tp / (tp + fn + 1e-8)
    specificity = tn / (tn + fp + 1e-8)
    dice = 2 * tp / (2 * tp + fp + fn + 1e-8)
    iou = tp / (tp + fp + fn + 1e-8)
    f1 = 2 * precision * sensitivity / (precision + sensitivity + 1e-8)

    return {
        "tp": int(tp),
        "fp": int(fp),
        "fn": int(fn),
        "tn": int(tn),
        "dice": float(dice),
        "iou": float(iou),
        "sensitivity": float(sensitivity),
        "specificity": float(specificity),
        "precision": float(precision),
        "f1": float(f1),
    }
```

12.9 Why accuracy is not enough

Retinal vessel and lesion pixels are typically much less common than background pixels. A system that predicts background almost everywhere can look accurate while missing the target structure. Therefore the report should prioritize overlap and class-specific metrics rather than accuracy alone.

12.10 Evaluation rules

- Confirm prediction and ground truth have identical dimensions.
- Apply the relevant FOV mask before computing DRIVE vessel metrics.
- Use development data to tune parameters; reserve held-out data for the final evaluation you describe as such.
- Use the same preprocessing and evaluation code for every experiment.
- Record exact parameter values with each experiment.
- Include failure cases instead of hiding them.

13. Stage G - Experiments and parameter tuning

13.1 Baseline vs improvements

A strong academic project shows that each processing stage has a measurable purpose. Build a simple baseline first, then add one technique at a time and compare metrics.

Experiment	Green	CLAHE	Gabor	Morphology	FOV	Purpose
E1	Yes	No	No	No	No	Simple intensity baseline
E2	Yes	Yes	No	No	No	Effect of local contrast
E3	Yes	Yes	Yes	No	No	Effect of oriented filtering
E4	Yes	Yes	Yes	Yes	No	Effect of cleanup
E5	Yes	Yes	Yes	Yes	Yes	Full baseline vessel pipeline

13.2 Gabor experiments

Parameter	Values to try
lambda	6, 8, 10, 12
sigma	2, 3, 4, 5
orientations	6, 12, 18
gamma	0.5, 0.7, 1.0
threshold	Otsu vs adaptive threshold

13.3 Exudate experiments

Experiment	Change
A1	No CLAHE
A2	CLAHE on luminance
A3	Top-hat without morphology
A4	Top-hat + morphology
A5	Optic-disc suppression in evaluation mode
A6	Different top-hat kernel sizes, e.g. 9, 15, 21
A7	Fixed threshold vs Otsu vs percentile threshold

13.4 Parameter selection procedure

22. Split available annotated development images into a tuning portion and a held-out portion.
23. Choose a small grid of reasonable values.
24. Run the complete pipeline for each configuration.
25. Compute the same metrics for every configuration.
26. Save the parameters and metrics into CSV.
27. Choose the configuration based on the predefined validation rule, not after looking at one favorite image.
28. Run the final chosen configuration on the held-out set once and report those results separately.

13.5 Experiment CSV

```
experiment_id,dataset,clahe,gabor_sigma,gabor_lambda,threshold,open,close,dice,iou,sensitivity,specificity,precision,f1
E1,DRIVE,1,4,10,otsu,1,1,ACTUAL,ACTUAL,ACTUAL,ACTUAL,ACTUAL,ACTUAL
```

Experiment integrity: The word ACTUAL above is a schema placeholder only. Never paste sample numbers into the final results table.

14. Stage H - Integrate the complete pipeline

14.1 Recommended public API

```
def preprocess(image):
    ...
    return result


def segment_vessels(image, fov_mask=None):
    ...
    return vessel_mask


def detect_exudates(image, optic_disc_mask=None):
    ...
    return exudate_mask, regions
```

```
def calculate_metrics(pred, gt):
    ...
    return metrics


def make_overlay(image, vessel_mask, exudate_mask):
    ...
    return overlay


def analyze_retina(image, fov_mask=None, optic_disc_mask=None):
    ...
    return result
```

14.2 Complete result object

```
result = {
    "original": image,
    "preprocessed_vessel": vessel_preprocessed,
    "gabor_response": gabor_response,
    "vessel_mask": vessel_mask,
    "exudate_tophat": tophat,
    "exudate_mask": exudate_mask,
    "regions": regions,
    "overlay": overlay,
    "vessel_density": vessel_density,
}
```

14.3 Why this design is useful

- The UI does not need to know how Gabor filtering works.
- The evaluation module can reuse the same segmentation functions.
- Intermediate images can be displayed in the debug view.
- The report generator can consume a single result dictionary.
- Unit tests can target one function at a time.

14.4 Command-line entry point

```
from pathlib import Path
import cv2

from src.pipeline import analyze_retina

img_bgr = cv2.imread("sample.jpg")
img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

result = analyze_retina(img)

Path("outputs/overlays").mkdir(parents=True, exist_ok=True)
cv2.imwrite(
    "outputs/overlays/sample_overlay.png",
    cv2.cvtColor(result["overlay"], cv2.COLOR_RGB2BGR)
)

print("Vessel density:", result["vessel_density"])
print("Candidate regions:", len(result["regions"]))
```

14.5 Integration checkpoint

- One function call produces all expected outputs.
- All intermediate arrays use predictable dtype and shape.
- No module imports Streamlit directly except app.py.
- No module depends on a specific hard-coded dataset filename.
- An output file is saved for every sample run.

15. Stage I - Build the Streamlit application

15.1 UI design

Screen	Contents
1. Upload	File uploader, image preview, dimensions, analyze button
2. Processing	Status messages for preprocessing, vessels, exudates, analysis
3. Results	Original, vessel mask, exudate mask, combined overlay
4. Measurements	Vessel density, region count, total area, largest area
5. Evaluation	Ground-truth metrics only when ground truth is available
6. Research View	Intermediate pipeline images
7. Report	Downloadable PDF

15.2 Starter app.py

```
import cv2
import numpy as np
import streamlit as st

from src.pipeline import analyze_retina

st.set_page_config(
    page_title="Retinal Image Analysis",
    layout="wide"
)

st.title("Retinal Image Analysis Platform")
st.caption("Classical DIP research / decision-support prototype")

uploaded = st.file_uploader(
    "Upload a retinal fundus image",
    type=["jpg", "jpeg", "png", "tif", "tiff"]
)

if uploaded is not None:
    raw = np.frombuffer(uploaded.read(), np.uint8)
    bgr = cv2.imdecode(raw, cv2.IMREAD_COLOR)

    if bgr is None:
        st.error("Could not decode the image")
        st.stop()

    image = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    st.image(image, caption="Original Fundus Image", use_container_width=True)

    h, w = image.shape[:2]
    st.write({"width": w, "height": h})

    if st.button("Analyze Retina"):
        with st.status("Processing retinal image..."):
            result = analyze_retina(image)

        c1, c2 = st.columns(2)
        with c1:
            st.image(result["vessel_mask"], caption="Vessel Mask")
        with c2:
            st.image(result["exudate_mask"], caption="Candidate Exudate Mask")

        st.image(result["overlay"], caption="Annotated Result", use_container_width=True)

        st.subheader("Quantitative Findings")
        st.metric("Candidate regions", len(result["regions"]))
        st.metric("Vessel density", f"{result['vessel_density']:.2f}%")

        st.info(
            "Candidate retinal regions detected. Clinical review is recommended where appropriate."
```

```
        )
```

15.3 Run the UI

```
streamlit run app.py
```

15.4 UI quality rules

- Use captions and legends so the colors in the overlay are unambiguous.
- Keep raw arrays and debug kernels out of the main results screen.
- Add a Research / Debug View for intermediate images.
- Display metrics with units or definitions where needed.
- Never display “DR positive” from this segmentation pipeline.
- Ensure an invalid upload gives a human-readable error instead of a traceback.

16. Stage J - Generate the PDF report

16.1 Report contents

29. Project title and report ID.
30. Image ID, dimensions, and acquisition source when known.
31. Original fundus image.
32. Preprocessed image(s).
33. Vessel segmentation mask.
34. Candidate exudate mask.
35. Combined overlay.
36. Candidate-region table.
37. Vessel density and other image-derived measurements.
38. Evaluation metrics when ground truth exists.
39. Processing pipeline diagram.
40. Limitations and failure cases.
41. Research / decision-support disclaimer.

16.2 Report generator skeleton

```
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Image, Table
from reportlab.lib.styles import getSampleStyleSheet


def generate_report(result, output_path):
    doc = SimpleDocTemplate(output_path, pagesize=A4)
    styles = getSampleStyleSheet()
    story = []

    story.append(Paragraph(
        "RETINAL IMAGE ANALYSIS REPORT",
        styles["Title"]
    ))
    story.append(Spacer(1, 10))

    # Add images, measurements, metrics and disclaimer here.
    # Use actual values from result.

    doc.build(story)
```

16.3 Example report summary wording

```
Analysis Summary

Candidate hard-exudate regions detected: <actual count>
Vessel density within analyzed FOV: <actual value>%
Largest candidate region: <actual area> pixels

Interpretation:
The system identifies image regions that satisfy the programmed
classical-DIP criteria. These outputs are intended for research /
decision-support visualization and are not a medical diagnosis.
```

16.4 What to include in the visual report

Report figure	Why it belongs
Original	Shows the source image
Green / L channel	Shows the signal selected by preprocessing
Gabor response	Explains how vessel-like structures were enhanced
Vessel mask	Shows final segmentation
Top-hat response	Explains bright-region enhancement
Candidate mask	Shows exudate candidates
Final overlay	Combines the two branches
Failure case	Shows what the algorithm gets wrong and why

17. Stage K - Testing, debugging, and failure cases

17.1 Unit tests

Test	Expected
read valid image	Array is non-null, 3 channels
invalid image	Clear error / False status
green channel	2D uint8 array
CLAHE	Same H x W as input
Gabor output	2D float array, same H x W
binary vessel mask	Values only 0 / 255
candidate mask	Values only 0 / 255
metrics	No NaN / infinity
overlay	Same H x W x 3 as RGB image
report	File exists and opens

17.2 Visual debugging checklist

- Original image: correct colors?
- Green channel: vessels visible?
- CLAHE: contrast improved without extreme noise?
- Gabor response: line structures visible?
- Vessel mask: reasonable structures, not mostly background?
- Top-hat: bright local structures highlighted?
- Candidate mask: too many tiny specks or large bright-disc false positive?
- Overlay: colors and alignment correct?

17.3 Common failure cases

Failure	Likely cause	Fix / experiment
Vessels missing	Threshold too strict; illumination correction too aggressive	Tune threshold, Gabor scale, preprocessing
Too many vessel false positives	Strong texture/noise	Adjust Gabor parameters and morphology
Optic disc marked as exudate	Bright-region detector has no disc handling	Use optic-disc handling in evaluation mode; implement automatic localization later
Too many tiny exudate candidates	Noise / top-hat kernel too small	Increase area filter or structuring element; compare thresholds
Large bright regions merged	Top-hat / morphology too aggressive	Reduce kernel or opening/closing
Metrics look impossible	Mask alignment / dtype / inversion error	Visualize pred and GT and inspect unique values
UI is slow	Processing full high-res images repeatedly	Cache immutable steps, resize for preview but preserve full-resolution evaluation

17.4 Metric sanity checks

- Prediction identical to ground truth -> Dice and IoU should be 1 (within numerical tolerance).
- Prediction all zeros with non-empty ground truth -> Dice should be near 0.
- Prediction all ones with non-empty ground truth -> overlap is non-zero but specificity should be poor.
- Pred/GT shape mismatch -> raise an error rather than broadcasting silently.

18. Stage L - Documentation, PPT, demo, and viva

18.1 README structure

42. Project title and one-paragraph abstract.
43. Problem statement.
44. Objectives.
45. Datasets and licensing.
46. Pipeline diagram.
47. Algorithms used.
48. Installation instructions.
49. How to run command-line pipeline.
50. How to run Streamlit.
51. Evaluation methodology.
52. Experiment table.
53. Results and failure cases.
54. Limitations and future work.
55. Disclaimer and references.

18.2 Suggested PPT sequence

Slide	Content
1	Title + team / guide
2	Problem statement
3	Motivation and objective
4	Dataset strategy
5	System architecture
6	Preprocessing

7	Vessel segmentation
8	Exudate candidate detection
9	Connected components and measurements
10	Evaluation metrics
11	Experiments
12	Results + example outputs
13	Streamlit demo
14	Limitations / future work
15	Conclusion

18.3 Demo script

56. Open the application.
57. Upload a retinal fundus image.
58. Show the original image.
59. Click Analyze Retina.
60. Explain preprocessing in one sentence.
61. Show vessel mask and explain Gabor orientations.
62. Show candidate exudate mask and explain top-hat.
63. Show connected-component region statistics.
64. Show the combined overlay.
65. Show evaluation metrics for a dataset image with ground truth.
66. Download and open the generated PDF report.
67. Finish with limitations and the non-diagnostic scope.

18.4 Viva questions you should be able to answer

Question	Core answer you should know
Why green channel?	Useful vessel/background contrast in many fundus images.
Why CLAHE?	Improves local contrast while limiting excessive amplification.
Why Gabor?	It responds to oriented, line-like structures such as vessels.
Why multiple orientations?	Vessels occur at many directions.
Why Otsu?	Provides an automatic baseline threshold from the response histogram.
Why opening?	Removes small isolated noise.
Why closing?	Bridges small gaps and connects nearby fragments.
Why top-hat?	Highlights bright local structures relative to a local morphological background.
Why connected components?	Converts a binary mask into individually measurable candidate regions.
Why FOV mask?	Excludes irrelevant pixels outside the retinal field.
Why optic-disc handling?	The optic disc is bright and can be a false positive for a bright-lesion detector.
Why Dice and IoU?	They quantify overlap with the ground-truth segmentation.
Why not accuracy alone?	Target pixels are a minority, so background dominance can make accuracy misleading.
Why two datasets?	DRIVE is designed for vessel extraction; IDRiD provides hard-exudate masks.
Why no deep learning?	The project specifically demonstrates explainable classical DIP techniques.
Is this a medical diagnosis tool?	No. It is a research / decision-support prototype.

What are limitations?	False positives, missed lesions, parameter sensitivity, image-quality variation, and lack of clinical validation.

19. Troubleshooting guide

19.1 Image appears blue / red

OpenCV reads BGR by default. Matplotlib and Streamlit generally expect RGB display order. Convert explicitly with cv2.cvtColor(..., cv2.COLOR_BGR2RGB).

19.2 Mask looks all black

- Print np.unique(mask).
- Print threshold value or Otsu result.
- Display the normalized response before thresholding.
- Check whether the response is float and whether it was normalized.
- Confirm you did not accidentally use an inverted mask.

19.3 Mask looks all white

- Check the response histogram.
- Check whether the threshold is too low.
- Check that background correction has not collapsed the dynamic range.
- Print the percentage of positive pixels.

19.4 Metrics are too good to be true

First suspect data leakage or a mask-processing bug, not a breakthrough. Common mistakes include evaluating the ground truth against itself, mismatched file names, accidental use of the same mask for prediction and ground truth, or applying a threshold after already loading the annotation as the prediction.

19.5 Streamlit keeps rerunning

Streamlit reruns the script after interactions. Keep expensive work inside the Analyze button block and use caching carefully for deterministic data where appropriate. Do not cache mutable image arrays in a way that can be accidentally modified.

19.6 High-resolution IDRiD is slow

Use a reduced-resolution preview for interactive debugging, but keep a separate full-resolution evaluation mode. Document any resize operation because resizing changes pixel geometry and the meaning of area-based measurements.

20. Final success checklist

20.1 Technical checklist

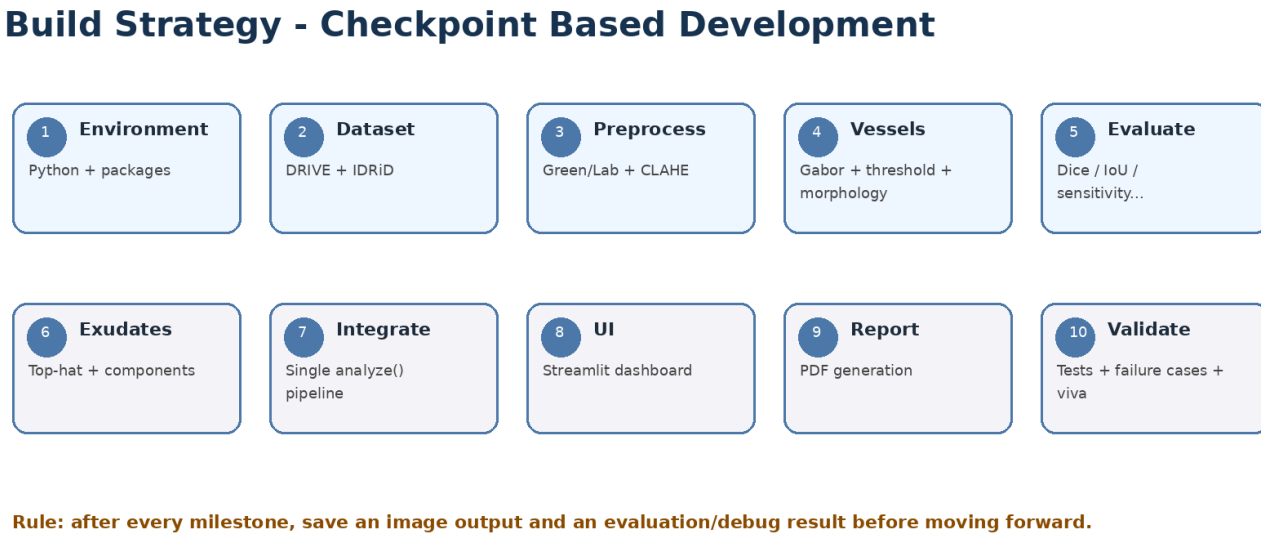
- [ ] Environment created and requirements.txt saved.
- [ ] DRIVE and IDRiD organized correctly.
- [ ] Image loader and validation work.
- [ ] FOV handling implemented.
- [ ] Preprocessing images saved and visually checked.
- [ ] Gabor vessel baseline works on sample images.

- [ ] Morphological cleanup works.
- [ ] Vessel masks saved.
- [ ] Vessel metrics computed from aligned masks.
- [ ] Exudate top-hat pipeline works.
- [ ] Connected components generate region statistics.
- [ ] Optic-disc issue is explicitly handled or documented.
- [ ] Combined overlay generated.
- [ ] Batch experiment script works.
- [ ] Results saved to CSV.
- [ ] Streamlit upload and analysis works.
- [ ] PDF report is generated.
- [ ] Invalid images show clean errors.
- [ ] Intermediate outputs are inspectable.

20.2 Academic checklist

- [ ] Problem statement is clear.
- [ ] Objective is measurable.
- [ ] Dataset choice is justified.
- [ ] Every major algorithm has a reason.
- [ ] Validation and test data are separated.
- [ ] Metrics are defined mathematically.
- [ ] Experimental comparisons are documented.
- [ ] Failure cases are shown.
- [ ] Limitations are stated honestly.
- [ ] No clinical-diagnosis claim is made.

20.3 Recommended execution order for maximum chance of success



68. Day / Session 1: environment + dataset viewer.
69. Day / Session 2: preprocessing + saved intermediate images.
70. Day / Session 3-4: vessel baseline + overlay.
71. Day / Session 5: vessel evaluation and parameter sweep.
72. Day / Session 6-7: exudate candidate pipeline + components.
73. Day / Session 8: exudate evaluation + optic-disc analysis.

74. Day / Session 9: integrated analyze_retina() pipeline.
75. Day / Session 10: Streamlit UI.
76. Day / Session 11: PDF report + experiment tables.
77. Day / Session 12: failure-case analysis, README, PPT, viva rehearsal.

> **Execution strategy:** The exact number of days depends on your available time. The important part is the order: every algorithmic block is tested before the UI is built around it.

21. References and official dataset sources

[1] DRIVE - Digital Retinal Images for Vessel Extraction. Official Grand Challenge website. Dataset description, image dimensions, training/test split, FOV masks, and evaluation information.

https://drive.grand-challenge.org/

[2] IDRiD - Indian Diabetic Retinopathy Image Dataset. Official Grand Challenge data page. Dataset size, pixel-level lesion subset, hard-exudate masks, optic-disc masks, image specifications, and fovea metadata.

https://idrid.grand-challenge.org/Data/

[3] IDRiD official challenge website. Background and challenge tasks, including lesion segmentation and optic disc / fovea detection.

https://idrid.grand-challenge.org/

[4] IDRiD rules / challenge documentation. Dataset citation and publication information.

https://idrid.grand-challenge.org/Rules/

Final project statement

The finished project should demonstrate a complete, explainable classical Digital Image Processing workflow: input validation -> preprocessing -> blood-vessel segmentation -> candidate hard-exudate detection -> region analysis -> quantitative evaluation -> visualization -> interactive application -> report generation. The strength of the project is not a claimed clinical diagnosis; it is the ability to explain each image-processing decision, reproduce the pipeline, compare alternatives with metrics, and clearly communicate limitations.

> **Final rule:** When you start implementing, build the project in checkpoints and keep the first baseline simple. Once every checkpoint is working, improve the algorithm experimentally rather than adding complexity blindly.